01 학습 방법과 코드 변경을 함께 읽기

MediFlow Skin · Web Skin · Hair
실제 구현과 저장 실험 기록을 대조한 상세 해설
2026-10-27

이 문서가 답하는 질문

CE·LS·Focal은 코드에서 무엇이 달라지는가? 2단계 학습은 단순한 추가 학습과 어떻게 다른가? ImageNet·DINOv2·MedSigLIP을 불러오면 정확히 무엇을 받는가? 논문 방법은 라이브러리 한 줄인지 직접 작성한 구조인지, 학습 뒤 실제 사진 한 장을 넣을 때도 추가 처리가 필요한지 설명한다.

읽는 관점	확인할 내용
불러오는 것	라이브러리, 구조, 사전학습 가중치, 전처리 설정, 우리 저장 모델을 구분
바꾸는 코드	손실 함수·trainable 설정·출력층·특징 결합·학습 반복을 실제 코드와 연결
얻은 결과	방법의 목적, 프로젝트에 적용한 범위, Validation 판단과 최종 Test를 구분
사용하는 방법	필요한 입력 크기·전처리·출력 결합과 외부 기본 모델 의존성을 설명

코드를 읽는 수준

Python을 처음 보는 팀원도 읽을 수 있도록 먼저 역할을 설명하고 실제 코드 발췌를 붙였다. 발췌 코드는 주변 함수·변수·입력 준비를 생략하므로 독립 실행용 노트북이 아니다. "설명용 축약 코드"는 동작을 풀어 쓴 예시이며 실제 발췌와 구분한다.

이 문서 작성으로 새 학습·추론·벤치마크를 실행하지 않았다. 현재 src의 구현과 당시 results에 보존된 소스·JSON을 대조했다. 최종 근거는 마지막의 파일·함수·해시 목록에서 찾는다.

02 먼저 구분할 여섯 종류의 변경

변경 종류	실험	코드에서 달라지는 곳
손실 함수	CE / LS / Focal	model.compile(loss=...)
학습 범위·순서	분류층 → 부분 미세조정	trainable, optimizer, model.fit
표현 학습	SupCon / MixStyle	특징용 손실, 증강, 중간 Layer
본체·사전학습	B0/B1/V2-S / DINOv2 / MedSigLIP	모델 생성자·preset·from_pretrained
영역·다중 크기	WS-DAN / PMG	attention, crop/drop, 분류 갈래와 결합
업데이트 방식	SAM	GradientTape 두 번, 가중치 교란과 복원

서로 조합할 수 있다

예를 들어 Hair v2의 "B1·384·LS 0.05·Adam·2단계"는 본체 B1, 입력 384, 손실 LS, 업데이트 Adam, 학습 범위 2단계를 함께 지정한 이름이다. LS를 선택한다고 B1 구조가 바뀌지 않고, 입력을 키운다고 CE가 다른 손실로 바뀌지도 않는다.

가중치와 학습의 관계

가중치는 모델이 이미 배운 숫자다. 사전학습 가중치는 출발점이며, 우리 Train으로 학습하면서 일부 가중치를 바꾸면 질환 분류 모델이 된다. Validation은 설정과 저장 시점 선택에 사용하고 Test는 후보를 고정한 뒤 평가하는 데이터다.

공통 경로: 사진 파일 → RGB 읽기·크기 조정 → 특징 추출 본체 → 새 분류층 → 클래스 점수 → 손실 계산 → 학습 가능한 가중치 업데이트. 학습이 끝나면 손실 계산과 업데이트를 제외하고 예측 경로를 실행한다.

03 import와 모델 다운로드는 다른 동작

코드 표현	불러오는 내용	우리 데이터까지 배웠나?
import keras	모델·Layer·손실·학습 도구를 제공하는 Python 라이브러리	아니다
EfficientNetB1(weights="imagenet")	B1 구조와 ImageNet에서 배운 가중치	아직 아니다
from_preset("dinov2_small")	DINOv2 구조·가중치 또는 변환 설정	우리 두피 라벨은 아직 아님
from_pretrained("google/medsiglip-448")	저장소의 모델 설정·사전학습 가중치 또는 processor 설정	우리 5클래스 분류층은 별도
load_model("stage2_best.keras")	우리 실험에서 저장한 구조·가중치와 저장된 학습 설정	해당 실험에서 학습한 모델

include_top=False의 의미

ImageNet의 기존 1,000개 분류용 마지막 층을 제외하고 특징 추출 부분을 가져온다는 뜻이다. 기존 본체가 피부질환 이름을 알고 있다는 뜻이 아니다. 우리 클래스 수에 맞는 새 Dense 층을 붙여 학습해야 한다.

첫 실행과 재실행

weights나 preset을 처음 요청하면 필요한 파일을 외부 저장소에서 받아 로컬 cache에 둘 수 있다. 이미 cache가 있으면 재사용할 수 있다. Google Drive의 데이터 ZIP을 읽는 작업과 외부 모델 가중치를 받는 작업은 서로 다르다.

HF_TOKEN은 접근 권한을 확인하는 자격 정보다. 질환 데이터, 모델 가중치, 학습 설정값이 아니며 문서나 발표에 실제 토큰 문자열을 넣지 않는다.

04 공통 EfficientNet을 만드는 실제 코드

실제 코드 발췌 | common_engine.py:119-140

```
def build_model(spec):
    builder = {
        "B0": keras.applications.EfficientNetB0,
        "B1": keras.applications.EfficientNetB1,
        "V2S": keras.applications.EfficientNetV2S,
    }
    size = spec["size"]
    backbone = builder[spec["backbone"]](
        include_top=False, weights="imagenet", input_shape=(size, size, 3)
    )
    backbone.trainable = False
    inputs = keras.Input((size, size, 3))
    features = backbone(inputs, training=False)
    features = keras.layers.GlobalAveragePooling2D()(features)
    features = keras.layers.Dropout(0.3)(features)
    outputs = keras.layers.Dense(spec["class_count"], activation="softmax")(features)
    model = keras.Model(inputs, outputs)
    model.compile(
        optimizer=keras.optimizers.Adam(1e-4),
        loss=loss_function(spec["loss"]),
        metrics=["accuracy"],
    )
```

코드	역할
input_shape=(size,size,3)	지정 크기의 RGB 세 채널 사진 입력
backbone.trainable=False	Stage 1에서 본체 가중치 업데이트 금지
backbone(..., training=False)	본체 내부 BN 등을 추론 모드로 호출. 가중치 동결과는 별개
GlobalAveragePooling2D	위치별 특징을 채널별 평균으로 요약
Dropout(0.3)	학습 중 일부 특징을 무작위로 비활성화하는 정규화
Dense(class_count, softmax)	우리 클래스 수만큼 출력하는 새 분류층

Adam은 가중치를 조정하는 optimizer이고 1e-4는 학습률이다. loss_function은 다음 페이지들의 CE/LS/Focal 중 하나를 돌려준다. 현재 저장 EfficientNet에 외부 /255를 중복 적용하지 않는다.

05 데이터는 어디서 들어오고 어떻게 바뀌나

실제 코드 발췌 | common_workflow.py:388-405

```
def factory(context, variant, seed=None):
    shuffle_seed = context["config"]["seed"] if seed is None else seed

    def load(split, size, shuffle):
        # All trials use exactly the same ORIGINAL validation and test images.
        root = context["roots"][variant if split == "train" else "original"]
        ds = keras.utils.image_dataset_from_directory(
            root / split,
            class_names=context["classes"],
            label_mode="categorical",
            image_size=(size, size),
            interpolation="bilinear",
            batch_size=context["config"]["batch_size"],
            shuffle=shuffle,
            seed=shuffle_seed if shuffle else None,
        )
        paths = [Path(p).relative_to(context["extracted"]).as_posix() for p in
    ds.file_paths]
        return ds.prefetch(tf.data.AUTOTUNE), paths
```

이 코드가 고정하는 것

Train만 variant에 따라 원본 또는 원본+증강 폴더를 선택한다. Validation/Test는 항상 original을 읽는다. class_names를 명시해 폴더 정렬 우연에 따라 출력 번호가 바뀌지 않게 한다. label_mode="categorical"은 정답을 클래스 수 길이의 one-hot 배열로 만든다는 뜻이다.

image_size와 bilinear 보간으로 입력 크기를 맞춘다. prefetch는 다음 batch를 미리 준비해 계산과 입력 대기를 겹치는 기능이며 새로운 사진을 생성하는 증강이 아니다.

전처리는 본체마다 다르다

EfficientNet 공통 경로는 float32 RGB 0~255를 전달하고 모델 내부 전처리를 사용한다. DINOv2는 preset image converter를, MedSigLIP은 AutoProcessor를 사용한다. 한 모델의 정규화를 다른 모델에 그대로 복사하면 안 된다.

06 CE는 기본 손실 함수를 선택한다

실제 코드 발췌 | common_engine.py:109-116

```
def loss_function(name):
    if name == "ce":
        return keras.losses.CategoricalCrossentropy()
    if name == "ls005":
        return keras.losses.CategoricalCrossentropy(label_smoothing=0.05)
    if name == "focal15":
        return keras.losses.CategoricalFocalCrossentropy(alpha=1.0, gamma=1.5)
    raise ValueError(name)
```

정답 점수가 낮을수록 큰 손실

CE(Cross Entropy)는 정답 클래스에 준 softmax 값을 p라고 할 때 -log(p)를 최소화한다. 같은 비듬 사진에서 비듬 점수 0.9와 0.2를 비교하면 0.2에 더 큰 손실을 준다. 이는 원리를 위한 예시이며 우리 실험 성능 숫자가 아니다.

설명용 축약 코드

```
model.compile(
    optimizer=keras.optimizers.Adam(1e-4),
    loss=keras.losses.CategoricalCrossentropy(),
    metrics=["accuracy"],
)
```

무엇을 불러왔나

Keras가 제공하는 손실 계산 객체를 생성한다. 별도 모델이나 가중치를 내려받지 않는다. loss만 바꾸면 B0/B1 본체와 출력 클래스 수는 그대로다. PMG는 softmax 전 점수인 logits에 CE를 적용하므로 from_logits=True를 쓴다. PyTorch CrossEntropyLoss도 logits를 입력받는다는 차이가 있다.

프로젝트에서 얻은 결과

Skin v1은 B0·224·CE·증강 조건으로 선정했다. Web Skin v1은 B0·256·CE였고 v2 PMG도 CE를 사용했다. CE는 비교 기준이면서 최종 후보에도 남은 손실이다.

07 2단계 학습은 학습 가능한 층을 바꾼다

실제 코드 발췌 | common_engine.py:144-161

```
def configure_partial(model, loss_name):
    backbones = [
        layer
        for layer in model.layers
        if isinstance(layer, keras.Model) and "efficientnet" in layer.name.lower()
    ]
    if len(backbones) != 1:
        raise ValueError("Expected one EfficientNet backbone")
    backbone = backbones[0]
    backbone.trainable = True
    for index, layer in enumerate(backbone.layers):
        layer.trainable = index >= len(backbone.layers) - 30 and not isinstance(
            layer, keras.layers.BatchNormalization
        )
    model.compile(
        optimizer=keras.optimizers.Adam(1e-5), loss=loss_function(loss_name),
    metrics=["accuracy"]
    )
    return [layer.name for layer in backbone.layers if layer.trainable]
```

코드의 변화

Stage 1은 공통 build_model에서 backbone.trainable=False다. Stage 2는 본체를 열고 마지막 30개 층 범위 중 BatchNormalization이 아닌 층만 trainable=True로 둔다. "30개를 정확히 모두 학습"한다는 뜻은 아니다. BN 등 제외 층 때문에 실제 학습 가능한 층 수는 달라질 수 있다.

compile을 다시 호출해 새 trainable 목록과 Adam(1e-5)를 반영한다. 학습 대상, 학습률, optimizer 상태와 학습량이 함께 달라지므로 상승 전체를 특정 한 변경의 인과 효과로 말하지 않는다.

실험군	Stage 1	Stage 2
대표 Clean Hair/Web Skin	분류층 15회	부분 미세조정 10회
Hair 논문 일반 분류·6클래스	분류층 15회	부분 미세조정 15회
Skin 원본/증강 비교	분류층 15회	없음
DINOv2 Linear Probe	고정 본체+새 분류층 15회	없음

08 best·last·추가 5회는 서로 다른 파일

실제 코드 발췌 | common_engine.py:176-186

```
def fit_stage(model, train, val, directory, name, epochs):
    best = directory / (name + "_best.keras")
    callbacks = [
        keras.callbacks.ModelCheckpoint(
            str(best), monitor="val_accuracy", mode="max", save_best_only=True
        ),
        keras.callbacks.CSVLogger(str(directory / (name + "_log.csv"))),
        HistoryBackup(directory / (name + "_history.json")),
        keras.callbacks.TerminateOnNaN(),
    ]
    history = model.fit(train, validation_data=val, epochs=epochs, callbacks=callbacks,
    verbose=2)
```

best와 last

best는 해당 단계에서 val_accuracy가 가장 높을 때 저장한 모델이다. last는 마지막 epoch가 끝난 상태다. 모델 선정에 쓰는 파일과 학습을 이어갈 출발 상태가 같지 않을 수 있다.

실제 코드 발췌 | common_engine.py:300-309

```
    parent_dir = Path(root) / parent["id"] / parent["attempt"]
    # Continue from epoch 10's LAST checkpoint, including optimizer state.
    source = parent_dir / "stage2_last.keras"
    model = keras.models.load_model(source)
    if model.optimizer is None:
        raise ValueError("Extension requires saved optimizer")
    train, _ = dataset_factory("train", 256, True)
    val, paths = dataset_factory("val", 256, False)
    started = time.monotonic()
    history, best = fit_stage(model, train, val, directory, "extension", epochs)
```

공통 extend_b1은 stage2_last.keras를 optimizer 상태까지 복원해 5회를 이어 학습한다. 새 최고가 부모 best를 넘어선 경우에만 연장 모델을 선택하고, 그렇지 않으면 부모 best를 보존한다. 단계 전환의 compile=False 재학습과 optimizer를 그대로 이어가는 연장은 다른 경로다.

Hair v1 연장은 개선되어 선정에 반영됐다. Web Skin 연장은 부모 최고를 넘지 못했다. 따라서 Web Skin 연장 행의 보존 성능 0.796을 새 5회의 성능 향상이라고 읽지 않는다.

09 LS 0.05는 정답 목표를 완화한다

실제 코드 발췌 | common_engine.py:112-113

```
    if name == "ls005":
        return keras.losses.CategoricalCrossentropy(label_smoothing=0.05)
```

코드는 한 옵션, 학습 목표는 변화

LS(Label Smoothing)는 CE를 계산할 때 one-hot 정답을 완화한다. Keras 방식은 y_smooth=(1-epsilon)*y+epsilon/K다. 5클래스에서 epsilon=0.05이면 정답 목표는 0.96, 나머지는 각각 0.01이다. 6클래스에서는 K가 달라져 같은 0.05라도 각 목표값이 달라진다.

설명용 축약 코드

```
target = [1, 0, 0, 0, 0]
# Five classes, epsilon = 0.05
smoothed = [0.96, 0.01, 0.01, 0.01, 0.01]
loss = keras.losses.CategoricalCrossentropy(
    label_smoothing=0.05
)
```

왜 사용했고 무엇은 바뀌지 않나

비슷한 두피 클래스에서 과도한 확신과 과적합을 완화할 수 있는지 시험했다. 입력 사진, 클래스 이름, 실제 정답은 그대로다. 목표 정확도를 95%로 제한하거나 사진의 라벨을 임의로 바꾸는 방식이 아니다. 예측 점수를 실제 정답 확률로 보정했다는 뜻도 아니다.

우리 결과

Hair B0·256에서 Validation은 CE보다 높았으나 Test의 일관된 개선으로 이어지지 않았다. 최종 Hair B1 후보들에 LS가 포함됐지만 개선 전부를 LS 하나의 효과로 설명하지 않는다. Web Skin에서는 CE를 교체할 이득을 확인하지 못했다. 근거: R01·R02.

10 Focal Loss는 사진별 비중을 바꾼다

실제 코드 발췌 | common_engine.py:114-115

```
    if name == "focal15":
        return keras.losses.CategoricalFocalCrossentropy(alpha=1.0, gamma=1.5)
```

실제로 선택한 설정

Keras CategoricalFocalCrossentropy(alpha=1.0, gamma=1.5)를 생성했다. alpha=1.0은 모든 클래스에 같은 계수를 쓰며, 이 설정의 핵심은 gamma에 따른 쉬운 표본 억제다. 클래스 불균형 가중치를 별도로 부여한 실험으로 설명하지 않는다.

설명용 축약 코드

```
CE = -log(p_true)
FL = alpha * (1 - p_true) ** gamma * CE
# Our loss comparison: alpha=1.0, gamma=1.5
```

정답 점수가 높은 사진은 (1-p_true)^gamma가 작아져 손실 비중이 줄어든다. 낮은 사진은 상대적 비중을 유지한다. gamma=0, alpha=1이면 이 조절 효과 없이 CE와 같은 형태가 된다.

우리 가설과 한계

클래스 장수가 비슷해도 미세각질·비듬처럼 계속 헷갈리는 표본은 있다. 이런 표본에 상대적으로 집중하면 구분력이 좋아질지 시험했다. 단, 어려운 표본에는 애매한 사진이나 라벨 오류도 섞일 수 있어 집중 자체가 항상 이득은 아니다.

원 논문은 객체 탐지의 쉬운 배경 불균형 문제를 다룬다. 우리 적용은 이 원리를 단일 사진 질환 분류에 옮긴 것이다. Hair는 당시 Test 선두 결과도 있었으나 Validation 선정 기준에서는 최종 후보가 아니었고 Web Skin에서도 CE보다 유리하지 않았다. 근거: R03.

11 B1·384와 EfficientNetV2-S를 바꾸는 곳

실제 코드 발췌 | paper_suite.py:60-79

```
    "efficientnetv2s_256": {
        "method": "standard",
        "backbone": "V2S",
        "size": 256,
        "loss": "ls005",
        "epochs1": 15,
        "epochs2": 15,
        "paper": "Tan and Le, ICML 2021",
        "hypothesis": "A newer efficient CNN backbone improves discriminative features",
    },
    "multires_b1_384": {
        "method": "standard",
        "backbone": "B1",
        "size": 384,
        "loss": "ls005",
        "epochs1": 15,
        "epochs2": 15,
        "paper": "Gessert et al., MethodsX 2020",
        "hypothesis": "Higher resolution preserves subtle scalp texture cues",
    },
```

설정은 어디에 반영되나

backbone="V2S"이면 common_engine의 생성자 표에서 keras.applications.EfficientNetV2S를 고른다. size=384는 사진 loader의 resize와 모델 input_shape에 함께 반영된다. 가중치가 같은 파일을 그대로 쓰는 것이 아니라 선택한 구조의 ImageNet 사전학습 가중치에서 새 실험을 시작한다.

384가 의미하는 것

256보다 원본 세부 정보가 덜 사라지는지 확인하는 입력 해상도 가설이다. 원본에 없는 정보를 만들어내지 않는다. 추론 시에도 해당 저장 모델의 입력 계약에 맞춰 384로 준비해야 한다. 크기를 256으로 바꾸면 같은 평가 조건이 아니다.

EfficientNetV2 논문의 전체 progressive learning이나 다중 해상도 피부 연구의 전체 앙상블을 재현하지 않았다. 프로젝트에서는 V2-S 구조 교체와 B1·384 고정 입력을 각각 선별했다. Hair는 B1·384 Adam을 선정했고 V2-S는 채택하지 않았다. 근거: R04·R05·R06.

12 SupCon은 별도 손실과 projector를 구현

실제 코드 발췌 | paper_suite.py:208-231

```
    backbone = keras.applications.EfficientNetB1(
        include_top=False,
        weights="imagenet",
        input_shape=(spec["size"], spec["size"], 3),
    )
    _partial_backbone(backbone)
    inputs = keras.Input((spec["size"], spec["size"], 3))
    features = backbone(inputs, training=False)
    pooled = keras.layers.GlobalAveragePooling2D(name="supcon_features")(features)
    feature_encoder = keras.Model(inputs, pooled, name="supcon_feature_encoder")
    projected = keras.layers.Dense(256, activation="relu")(pooled)
    projected = keras.layers.Dense(128)(projected)
    encoder_projector = keras.Model(inputs, projected, name="supcon_encoder_projector")
    augmenter = keras.Sequential(
        [
            keras.layers.RandomFlip("horizontal", seed=spec["seed"]),
            keras.layers.RandomRotation(0.03, seed=spec["seed"] + 1),
            keras.layers.RandomZoom(0.05, seed=spec["seed"] + 2),
            keras.layers.RandomContrast(0.1, seed=spec["seed"] + 3),
        ],
        name="weak_medical_views",
    )
    trainer = SupConTrainer(encoder_projector, augmenter)
    trainer.compile(optimizer=keras.optimizers.Adam(1e-5))
```

외부에서 받은 것과 직접 만든 것

외부에서 받은 부분은 ImageNet EfficientNet-B1 본체다. Dense 256→128 projector, 두 view 생성, 지도 대조 손실과 train_step은 프로젝트 코드로 작성했다. SupCon 이름으로 학습된 두피 완성 모델을 다운로드한 것이 아니다.

본체 후반부는 대조학습 시작부터 부분 학습 상태다. RandomFlip·Rotation·Zoom·Contrast로 한 사진의 두 view를 만들고 같은 라벨을 붙인다. 이미 저장된 오프라인 증강 Train에 학습 중 view 변환이 추가된 경로다. 근거: R07.

13 SupCon의 유사도와 분류 단계

실제 코드 발췌 | paper_suite.py:136-151

```
def supervised_contrastive_loss(labels, projections, temperature=0.1):
    """Khosla et al. loss; two views ensure every anchor has a positive."""
    labels = tf.argmax(labels, axis=1, output_type=tf.int32)
    projections = tf.math.l2_normalize(projections, axis=1)
    logits = tf.matmul(projections, projections, transpose_b=True) / temperature
    count = tf.shape(logits)[0]
    self_mask = tf.eye(count, dtype=tf.bool)
    logits = logits - tf.reduce_max(logits, axis=1, keepdims=True)
    exp_logits = tf.exp(logits) * tf.cast(~self_mask, logits.dtype)
    log_prob = logits - tf.math.log(tf.reduce_sum(exp_logits, axis=1, keepdims=True) +
    1e-12)
    positive = tf.equal(labels[:, None], labels[None, :]) & ~self_mask
    positive_count = tf.reduce_sum(tf.cast(positive, logits.dtype), axis=1)
    mean_log_prob = tf.reduce_sum(
        tf.cast(positive, logits.dtype) * log_prob, axis=1
    ) / tf.maximum(positive_count, 1.0)
    return -tf.reduce_mean(mean_log_prob)
```

손실이 보는 대상

특징을 길이 1로 정규화하고 모든 쌍의 내적을 temperature=0.1로 조절한다. 자기 자신은 제외하고 같은 클래스인 다른 특징을 positive로 사용한다. 정답 softmax 하나를 평가하는 CE와 달리 특징 사이 관계를 직접 학습한다.

실제 코드 발췌 | paper_suite.py:248-258

```
    classifier_inputs = keras.Input((spec["size"], spec["size"], 3))
    classifier_features = feature_encoder(classifier_inputs, training=False)
    classifier_features = keras.layers.Dropout(0.3)(classifier_features)
    classifier_outputs = keras.layers.Dense(
        spec["class_count"], activation="softmax"
    )(classifier_features)
    classifier = keras.Model(classifier_inputs, classifier_outputs)
    classifier.compile(
        optimizer=keras.optimizers.Adam(1e-5),
        loss=engine.loss_function(spec["loss"]),
        metrics=["accuracy"],
```

대조학습 15회 뒤 projector는 최종 분류 경로에서 제외하고 원래 feature_encoder에 Dropout과 softmax 분류층을 붙여 15회 학습한다. 중요한 점: 이 코드에는 feature_encoder를 새로 동결하는 줄이 없으므로 분류 단계에도 기존 본체 후반 trainable 상태가 유지된다. 순수 고정 특징 Linear Probe와 구분한다.

SupCon은 B1·256 기준선보다 Validation이 좋아졌지만 B1·384 최종 후보를 넘지 못했다. 실제 사용은 한 장 입력이며 학습용 두 view를 다시 준비할 필요가 없다.

14 DINOv2는 preset과 전처리를 함께 가져온다

실제 코드 발췌 | paper_suite.py:293-312

```
def _build_dinov2_classifier(spec):
    import keras_hub

    converter = keras_hub.layers.DINOV2ImageConverter.from_preset(
        spec["preset"],
        image_size=(spec["size"], spec["size"]),
        crop_to_aspect_ratio=False,
    )
    backbone = keras_hub.models.DINOV2Backbone.from_preset(
        spec["preset"], image_shape=(spec["size"], spec["size"], 3)
    )
    backbone.trainable = False
    feature_model = keras.Model(
        inputs=backbone.inputs,
        outputs=list(backbone.pyramid_outputs.values())[-1],
        name="dinov2_final_features",
    )
    inputs = keras.Input((spec["size"], spec["size"], 3), name="raw_rgb_0_255")
    converted = converter(inputs)
    features = feature_model({"images": converted}, training=False)
```

받은 항목	포함 내용
DINOV2Backbone.from_preset	자기지도 사전학습된 DINOv2 Small 본체·가중치
DINOV2ImageConverter.from_preset	해당 preset의 resize·정규화 등 영상 변환 설정
새로 붙인 Dense	우리 두피 5클래스 분류층. 처음부터 학습

DINOv2 본체는 고정하고 우리 라벨로 분류층을 15회 학습했다. 자기지도는 외부 본체의 사전학습 방식이며 우리 분류층을 무라벨로 학습했다는 뜻이 아니다. 입력은 raw RGB 0~255를 받되 converter가 DINOv2용 처리로 바꾼다. EfficientNet의 전처리 규칙을 그대로 옮기지 않는다. 근거: R08.

15 DINOv2의 token 출력과 차원 오류 수정

실제 코드 발췌 | paper_suite.py:313-339

```
    # DINOv2 emits [batch, tokens, channels], not a CNN feature map. Following
    # the official linear-classifier representation, combine the CLS token
    # with the mean of the patch tokens.
    token_count = int(features.shape[1])
    class_token = keras.layers.Cropping1D(
        cropping=(0, token_count - 1), name="keep_cls_token"
    )(features)
    class_token = keras.layers.Flatten(name="flatten_cls_token")(class_token)
    patch_tokens = keras.layers.Cropping1D(
        cropping=(1, 0), name="remove_cls_token"
    )(features)
    patch_mean = keras.layers.GlobalAveragePooling1D(name="mean_patch_tokens")(
        patch_tokens
    )
    features = keras.layers.Concatenate(name="cls_and_mean_patch")(
        [class_token, patch_mean]
    )
    features = keras.layers.Dropout(0.3)(features)
    outputs = keras.layers.Dense(spec["class_count"], activation="softmax")(features)
    model = keras.Model(inputs, outputs)
    model.compile(
        optimizer=keras.optimizers.Adam(1e-3),
        loss=engine.loss_function(spec["loss"]),
        metrics=["accuracy"],
        # Keras 3 may enable XLA automatically on GPU. DINOV2ImageConverter
        # uses ResizeBicubic, which the Colab XLA_GPU_JIT path does not support.
        jit_compile=False,
```

왜 GlobalAveragePooling2D가 실패했나

CNN은 보통 [batch,height,width,channels] 형태의 4차원 특징을 내지만 DINOv2는 [batch,tokens,channels] 형태의 3차원 특징을 냈다. 당시 오류의 (None,257,384)를 2D pooling에 넣으면 차원이 맞지 않는다.

현재 구현은 CLS token과 나머지 patch token의 평균을 결합한 뒤 분류한다. 실제 token 개수는 실행 시 출력 형태에서 읽는다. converter의 ResizeBicubic과 Colab XLA 경로 충돌을 피하기 위해 현재 코드에는 jit_compile=False가 있다. 이는 학습 이론 변경이 아니라 실행 호환성 조치다.

이번 고정 본체·224·분류층 설정의 Validation 지표가 기준선보다 낮아 채택하지 않았다. DINOv2 전체 미세조정이 실패했다는 결론은 아니다.

16 SAM은 직접 작성한 두 번의 gradient 계산

실제 코드 발췌 | hair_sam.py:105-128

```
    def train_step(images, labels):
        with tf.GradientTape() as first_tape:
            first_predictions = model(images, training=True)
            first_loss = loss_function(labels, first_predictions)
        first_gradients = first_tape.gradient(first_loss, variables)
        usable = [g for g in first_gradients if g is not None]
        norm = tf.linalg.global_norm(usable)
        perturbations = []
        for variable, gradient in zip(variables, first_gradients, strict=True):
            if gradient is None:
                perturbations.append(None)
                continue
            perturbation = tf.cast(rho, gradient.dtype) * gradient / (
                tf.cast(norm, gradient.dtype) + tf.cast(1e-12, gradient.dtype)
            )
            variable.assign_add(perturbation)
            perturbations.append(perturbation)
        with tf.GradientTape() as second_tape:
            predictions = model(images, training=True)
            loss = loss_function(labels, predictions)
        gradients = second_tape.gradient(loss, variables)
        for variable, perturbation in zip(variables, perturbations, strict=True):
            if perturbation is not None:
                variable.assign_sub(perturbation)
```

업데이트 순서

첫 GradientTape로 현재 손실의 gradient를 구하고, rho=0.05 범위의 교란을 가중치에 잠시 더한다. 두 번째 gradient를 계산한 뒤 교란을 빼서 원래 가중치를 복원하고 두 번째 gradient로 Adam(1e-5) 업데이트를 한다.

외부 SAM 완성 모델을 불러온 것이 아니다. 같은 B1·384 Stage 1 체크포인트를 로드하고 업데이트 절차만 비교했다. loss는 LS 0.05이고 미세조정 범위·15회·데이터를 유지했다. 학습 중 두 번 계산하지만 저장된 모델은 일반 B1 분류 경로여서 추론에 SAM 업데이트를 하지 않는다.

Adam과 Validation 정답 수는 997/1,252로 같고 Macro F1은 SAM이 더 낮았다. 따라서 Adam을 유지했다. "평평한 해"는 가중치 주변 손실에 관한 목적이며 실제 다른 촬영 장비에 강하다고 입증한 결과는 아니다. 근거: R09.

17 WS-DAN은 attention 구조를 직접 붙인다

실제 코드 발췌 | web_skin_wsdan.py:105-124

```
def build_model(class_count, size=256, attention_maps=8, weights="imagenet"):
    backbone = keras.applications.EfficientNetB0(
        include_top=False, weights=weights, input_shape=(size, size, 3)
    )
    backbone.trainable = False
    inputs = keras.Input((size, size, 3), name="image")
    features = backbone(inputs, training=False)
    attention = keras.layers.Conv2D(
        attention_maps, 1, activation="sigmoid", name="attention_maps"
    )(features)
    pooled = BilinearAttentionPooling(name="bilinear_attention_pooling")(
        [features, attention]
    )
    pooled = keras.layers.Dropout(0.3, name="attention_dropout")(pooled)
    outputs = keras.layers.Dense(class_count, activation="softmax", name="predictions")(
        pooled
    )
    model = keras.Model(inputs, outputs, name="web_skin_wsdan_b0")
    attention_probe = keras.Model(inputs, [outputs, attention], name="attention_probe")
    return model, attention_probe
```

무엇을 추가했나

ImageNet B0가 만든 특징 위에 1×1 Conv2D로 attention map 8개를 만든다. BilinearAttentionPooling은 각 map을 가중치로 특징을 모아 하나의 긴 배열로 결합하고 정규화한다. 여기에 새 분류층을 붙인다.

BilinearAttentionPooling과 attention crop/drop, 학습 loop는 프로젝트의 직접 구현이다. 이미지 클래스 라벨만 사용하므로 병변 위치 상자를 별도로 다운로드한 것이 아니다. 강조 영역을 의사의 정답 병변 위치로 간주하지 않는다.

Hu 등의 WS-DAN 아이디어를 EfficientNet-B0에 응용했다. 해당 원 논문의 전체 데이터·backbone·실험 프로토콜을 동일 재현한 것으로 표현하지 않는다. 근거: R10.

18 WS-DAN은 학습과 추론 경로도 달라진다

실제 코드 발췌 | web_skin_wsdan.py:249-259

```
    @tf.function
    def train_step(images, labels):
        with tf.GradientTape() as tape:
            raw_predictions, maps = attention_probe(images, training=True)
            augmented = attention_augment(
                images, maps, size, crop_threshold, drop_threshold
            )
            augmented_predictions = model(augmented, training=True)
            raw_loss = loss_function(labels, raw_predictions)
            augmented_loss = loss_function(labels, augmented_predictions)
            loss = 0.5 * (raw_loss + augmented_loss)
```

실제 코드 발췌 | web_skin_wsdan.py:220-226

```
    def __call__(self, images, training=False):
        raw, maps = self.probe(images, training=False)
        cropped_images = attention_crop(
            images, maps, self.size, self.crop_threshold
        )
        cropped = self.model(cropped_images, training=False)
        return (raw + cropped) / 2.0
```

Train과 Validation을 읽는 법

Train은 원본 CE와 attention 증강 CE를 0.5씩 평균하고 Train Accuracy는 원본 예측을 기록한다. crop/drop 선택은 학습 중 무작위다. Validation은 원본과 attention crop의 예측을 평균한 뒤 그 결과의 CE·Accuracy를 구한다. 같은 Loss 산식처럼 Train/Val의 간격을 비교하지 않는다.

사용자는 원본 사진 한 장만 제공한다. 내부에서는 원본 분석으로 attention을 만들고 확대 영역을 다시 분석하므로 두 번의 모델 실행이 필요하다. 이번 Validation Accuracy는 0.764로 저장 기준선 0.796보다 낮았고 선정하지 않았다.

19 PMG는 중간 특징 세 개와 fusion을 구성

실제 코드 발췌 | web_skin_paper_suite.py:110-122

```
    feature_model = keras.Model(
        backbone.input,
        [
            backbone.get_layer("block3a_expand_activation").output,
            backbone.get_layer("block5a_expand_activation").output,
            backbone.get_layer("top_activation").output,
        ],
        name="pmg_backbone",
    )
    feature_model.trainable = False
    inputs = keras.Input((size, size, 3), name="image")
    features = feature_model(inputs, training=False)
    embeddings, logits = [], []
```

실제 코드 발췌 | web_skin_paper_suite.py:130-143

```
        value = keras.layers.GlobalAveragePooling2D(
            name=f"pmg_branch{index}_pool"
        )(value)
        value = keras.layers.Dense(
            256, activation="elu", name=f"pmg_branch{index}_embedding"
        )(value)
        embeddings.append(value)
        logits.append(
            keras.layers.Dense(class_count, name=f"pmg_branch{index}_logits")(value)
        )
    combined = keras.layers.Concatenate(name="pmg_concat")(embeddings)
    combined = keras.layers.Dense(512, activation="elu", name="pmg_fusion")(combined)
    logits.append(keras.layers.Dense(class_count, name="pmg_concat_logits")(combined))
    return keras.Model(inputs, logits, name="web_skin_pmg_" + backbone_name.lower())
```

라이브러리와 자체 구현의 경계

불러온 것은 ImageNet B0/B1 본체다. 서로 다른 깊이의 특징 세 개를 꺼내고 각 갈래에 Conv·pooling·Dense를 붙인 구조, jigsaw 처리, 순차 학습 loop, 출력 결합은 직접 작성했다. PMG용 두피·피부 완성 가중치를 가져온 방식이 아니다.

초기 B0 실험 소스는 생성자를 EfficientNetB0로 고정했다. 이후 B1·384 비교용 현재 함수는 B0/B1을 선택하도록 일반화했지만 같은 세 지점의 특징과 세 branch+fusion 구성을 유지한다. 원 논문 아이디어를 현재 본체에 응용한 구현이다. 근거: R11.

20 PMG는 네 순차 업데이트와 합산 추론

실제 코드 발췌 | web_skin_paper_suite.py:203-217

```
        for images, labels in train:
            losses = [
                branch_step(images, labels, 0, 8),
                branch_step(images, labels, 1, 4),
                branch_step(images, labels, 2, 2),
            ]
            fusion_loss, predictions = fusion_step(images, labels)
            train_loss.update_state(tf.add_n(losses) + fusion_loss)
            train_accuracy.update_state(labels, predictions)
        val_loss = keras.metrics.Mean()
        val_accuracy = keras.metrics.CategoricalAccuracy()
        for images, labels in val:
            outputs = model(images, training=False)
            val_loss.update_state(loss_function(labels, outputs[3]))
            val_accuracy.update_state(labels, predictor(images, training=False))
```

실제 코드 발췌 | web_skin_paper_suite.py:156-159

```
def _pmg_predictor(model):
    total = keras.layers.Add(name="pmg_logit_sum")(model.outputs)
    probabilities = keras.layers.Activation("softmax", name="predictions")(total)
    return keras.Model(model.input, probabilities, name="web_skin_pmg_inference")
```

왜 시간이 더 들고 Loss가 다른가

한 batch에서 8×8·4×4·2×2 조각을 섞은 사진으로 각 branch를 한 번씩 학습하고, 원본에서 fusion을 한 번 더 학습한다. Train Loss는 세 branch CE+2×fusion CE의 합이고 Validation Loss는 fusion CE 하나다. 일반 CE 한 번과 절대 크기를 직접 비교하지 않는다.

중요 정정: 실제 _fit_pmg_stage의 fusion_step은 네 logits를 합한 softmax를 돌려주므로 Train Accuracy도 합산 예측 기준이다. Validation Accuracy 역시 네 출력 합산 기준이다. 이전 일부 설명의 "Train Accuracy=fusion 단독" 표현은 코드와 달라 이 문서에서 바로잡았다.

실제 입력은 원본 한 장이고 조각 섞기는 학습용이다. 다만 네 출력이 있는 저장 모델에 단일 softmax 모델과 같은 추론 코드를 쓰면 안 된다. PMG B0·256을 v2로 고정했고 B1·384는 Validation 선두 연구 조건으로 남겼다.

21 MixStyle은 특징 통계를 섞는 Layer

실제 코드 발췌 | web_skin_paper_suite.py:309-328

```
    def call(self, inputs, training=None):
        if training is not True:
            return inputs
        mean = tf.reduce_mean(inputs, axis=(1, 2), keepdims=True)
        variance = tf.reduce_mean(tf.square(inputs - mean), axis=(1, 2), keepdims=True)
        deviation = tf.sqrt(variance + 1e-6)
        normalized = (inputs - tf.stop_gradient(mean)) / tf.stop_gradient(deviation)
        permutation = tf.random.shuffle(tf.range(tf.shape(inputs)[0]))
        mixed_mean = tf.gather(mean, permutation)
        mixed_deviation = tf.gather(deviation, permutation)
        first = tf.random.gamma((tf.shape(inputs)[0], 1, 1, 1), self.alpha)
        second = tf.random.gamma((tf.shape(inputs)[0], 1, 1, 1), self.alpha)
        weight = first / (first + second + 1e-6)
        target_mean = weight * mean + (1.0 - weight) * mixed_mean
        target_deviation = weight * deviation + (1.0 - weight) * mixed_deviation
        styled = normalized * tf.stop_gradient(target_deviation) + tf.stop_gradient(
            target_mean
        )
        return tf.cond(
            tf.random.uniform(()) < self.probability, lambda: styled, lambda: inputs
```

어디에 삽입했나

EfficientNetB0의 block2b_add까지 early, 이후를 late로 나누고 그 사이에 MixStyle을 넣었다. 공간축 평균·표준편차를 구해 특징을 정규화하고, batch 안 다른 사진의 통계와 섞은 뒤 다시 씌운다. alpha=0.1의 Beta 혼합은 두 Gamma 표본으로 구현했다. 적용 확률은 0.5다.

training이 True가 아니면 입력을 그대로 반환하므로 실제 추론에는 스타일 혼합이 없다. MixStyle Layer는 자체 코드이고 사전학습 B0만 외부에서 받았다. 현재 코드의 jit_compile=False는 RandomGamma의 XLA 호환성 조치이며 당시 첫 보존 소스에는 없던 차이다.

이 공개 Validation에서는 기준선보다 좋아졌지만 PMG보다 낮았다. 실제 다른 웹캠·조명의 독립 강건성을 측정한 결과로 확대하지 않는다. 근거: R12.

22 MedSigLIP은 모델과 processor를 받는다

실제 코드 발췌 | web_skin_medsiglip_linear.py:127-142

```
def _load_encoder(token, model_id):
    import torch
    from transformers import AutoModel, AutoProcessor

    if not torch.cuda.is_available():
     raise RuntimeError("MedSigLIP 특징 추출에는 Colab GPU가 필요합니다.")
    processor = AutoProcessor.from_pretrained(model_id, token=token)
    model = AutoModel.from_pretrained(
        model_id,
        token=token,
        torch_dtype=torch.float16,
        low_cpu_mem_usage=True,
    ).eval().to("cuda")
    if not hasattr(model, "get_image_features"):
     raise TypeError("불러온 모델에 get_image_features가 없습니다.")
    return model, processor
```

불러온 것	실제로 들어 있는 내용
AutoModel	google/medsiglip-448의 설정과 사전학습 모델 가중치. 영상·텍스트 구성의 기본 모델
AutoProcessor	영상 크기 조정·정규화와 텍스트 처리 등 해당 모델의 입력 규칙
get_image_features	기본 모델의 영상 특징 배열을 얻는 함수. 이번 실험은 이 영상 경로를 사용
eval / inference_mode	추론 동작과 gradient 비활성화. 기본 모델 가중치를 학습하지 않음

Keras CNN 경로와 달리 PyTorch·Transformers를 쓴다. torch_dtype=float16, CUDA에서 특징을 추출한다. 모델 cache에 파일이 내려와도 그것은 우리 5클래스 최종 모델이 아니다. 의료 사전학습은 외부에서 완료됐고 우리는 그 특징의 분리 가능성을 시험했다. 근거: R13.

23 MedSigLIP의 실제 학습 대상은 Linear 층

실제 코드 발췌 | web_skin_medsiglip_linear.py:169-176

```
            inputs = processor(images=images, return_tensors="pt")
            pixel_values = inputs["pixel_values"].to("cuda", dtype=torch.float16)
            with torch.inference_mode(), torch.autocast("cuda", dtype=torch.float16):
                features = model.get_image_features(pixel_values=pixel_values)
                features = torch.nn.functional.normalize(features.float(), dim=-1)
            pieces.append(features.cpu().numpy())
        embeddings = np.concatenate(pieces).astype(np.float32, copy=False)
        _save_shard(shard_path, embeddings, labels[start:stop], relative[start:stop])
```

실제 코드 발췌 | web_skin_medsiglip_linear.py:189-198

```
    torch.manual_seed(context["config"]["seed"])
    torch.cuda.manual_seed_all(context["config"]["seed"])
    device = "cuda"
    head = torch.nn.Linear(train_x.shape[1], len(context["classes"])).to(device)
    optimizer = torch.optim.AdamW(
        head.parameters(),
        lr=context["config"]["linear_learning_rate"],
        weight_decay=context["config"]["linear_weight_decay"],
    )
    criterion = torch.nn.CrossEntropyLoss()
```

특징 cache와 저장 파일

특징을 L2 정규화해 NumPy 배열로 저장하고 128장 단위 shard cache를 사용한다. cache는 사진을 표현한 숫자이며 새로운 영상이나 모델 가중치가 아니다. 특징 추출 batch는 8, Linear 학습 batch는 256, AdamW 학습률 1e-3·weight decay 1e-4·50회다. Linear의 출력 수는 해당 도메인 클래스 수다.

medsiglip_linear_head.pt에는 분류층 state_dict·입력 차원·클래스 순서·model_id를 저장한다. 이 작은 파일 하나에 MedSigLIP 본체가 들어 있는 것은 아니다. 새 사진 추론에는 같은 기본 모델+processor+특징 정규화+저장 Linear를 함께 사용해야 한다.

Hair와 Web Skin 모두 frozen Linear 조건에서 현재 후보보다 Validation Macro F1이 낮아 Test·최종 후보 패키징을 하지 않았다. 생성형 문장 작성이나 전체 본체 미세조정 실험이 아니다.

24 Hair 6클래스는 출력과 데이터도 바뀐다

실제 코드 발췌 | hair_six_class_suite.py:288-312

```
        for label in CLASSES:
            folders = [root / "original" / split / label]
            if is_train and variant == "augmented":
                folders.append(root / "augmentation" / "train" / label)
            for folder in folders:
                files = [p for p in sorted(folder.iterdir()) if p.is_file()]
                paths.extend(str(p) for p in files)
                labels.extend([class_to_index[label]] * len(files))
        if not paths or (is_train and set(labels) != set(range(len(CLASSES)))):
            raise ValueError("Missing dataset images or class")
        labels_array = np.asarray(labels, dtype=np.int32)
        dataset = tf.data.Dataset.from_tensor_slices((paths, labels_array))
        if is_train:
            dataset = dataset.shuffle(len(paths), seed=seed, reshuffle_each_iteration=True)
            counts = np.bincount(labels_array, minlength=len(CLASSES))
            weights = np.sqrt(counts.mean() / counts)
            weights /= np.mean(weights[labels_array])

        def decode(path, label):
            image = tf.io.decode_image(tf.io.read_file(path), channels=3,
    expand_animations=False)
            image.set_shape((None, None, 3))
            image = tf.image.resize(tf.cast(image, tf.float32), (size, size))
            target = tf.one_hot(label, len(CLASSES))
            if is_train:
                return image, target, tf.gather(tf.constant(weights, tf.float32), label)
```

네 조건의 실제 변경

기존 다섯 클래스 뒤에 양호를 index 5로 붙인다. 새 Dense 출력은 6개이며 기존 5출력 저장 모델을 이어 학습하지 않고 ImageNet 본체에서 다시 시작했다. Train 원본은 10,460장, 증강 조건은 원본+7,155장으로 17,615장, Validation/Test는 각각 원본 1,305장이다.

weights=sqrt(클래스 평균 수량/해당 클래스 수량)를 계산하고 학습 표본별 평균이 1이 되도록 정규화한 sample weight를 준다. 따라서 원본/증강 비교에는 사진 수뿐 아니라 계산된 클래스 가중치 변화도 동반된다.

B0·256은 CE, B1·384는 LS 0.05이므로 B0/B1 차이는 본체·입력·손실이 함께 바뀐 구성 비교다. checkpoint 저장은 Validation Accuracy, 네 실행 중 성능 우선 후보는 Validation Macro F1 우선으로 선정했다.

25 최종 사용 때는 입력·출력 계약을 지킨다

저장 후보	입력·전처리	출력 처리
Skin v1 B0·224	float32 RGB 0~255, 224×224	10개 softmax 직접 사용
Web Skin v1 B0·256	float32 RGB 0~255, 256×256	5개 softmax 직접 사용
Web Skin v2 PMG B0·256	float32 RGB 0~255, 256×256	4개 logits 출력 합 → softmax
Hair5 v1 B1·256	float32 RGB 0~255, 256×256	5개 softmax 직접 사용
Hair5 v2 B1·384	float32 RGB 0~255, 384×384	5개 softmax 직접 사용
Hair6 B1·384 / B0·256	각 입력 크기, float32 RGB 0~255	6개 softmax, 양호 index 5

학습 방법이 추론에 남는 범위

CE·LS·Focal·SAM은 학습 방식이므로 추론 시 손실이나 gradient를 계산하지 않는다. SupCon의 projector·두 view도 최종 분류 경로에 없다. MixStyle은 비활성화된다. WS-DAN은 원본+crop 두 예측이 필요하고 PMG는 네 출력 합산이 필요하다. DINOv2는 converter, MedSigLIP은 외부 기본 모델과 저장 head가 필요하다.

class_names.json 배열 순서는 출력 index의 계약이다. 점수가 낮다고 임의로 특정 클래스에 치환하지 않고, 범위 밖·애매한 입력의 처리 정책은 별도로 검증한다. 이 문서의 입력 장비는 목표 사용 방식이며 공개 데이터 Test를 실제 촬영 성능으로 바꾸어 부르지 않는다.

26 저장 Validation 결과를 그대로 비교

다음 값은 학습곡선의 최고점이 아니라 저장한 선정 체크포인트의 재평가 결과다. Hair는 원본 Validation 1,252장, Web Skin은 원본 Validation 500장이다. 두 도메인을 하나의 성능 순위로 합치지 않는다.

Hair 5클래스

조건	Accuracy	Macro F1
B1·256 분류 기준선	0.7755591054313099	0.7746028470442391
SupCon B1·256	0.7915335463258786	0.7910880666407202
DINOv2 Small·224	0.7699680511182109	0.7705341251195019
EfficientNetV2-S·256	0.7691693290734825	0.7681559382466019
B1·384 Adam	0.7963258785942492	0.7957146810115047
B1·384 SAM	0.7963258785942492	0.7954748920799393
MedSigLIP Linear	0.7699680511182109	0.7687534259685547

Web Skin

조건	Accuracy	Macro F1
기존 B0·256·CE	0.796	0.7915394647566528
WS-DAN B0·256	0.764	0.7546949301321499
PMG B0·256	0.85	0.8473279632397033
MixStyle B0·256	0.806	0.8033767539683495
PMG B1·384	0.866	0.8643216544321994
MedSigLIP Linear	0.824	0.8195774288599683

수치는 validation_metrics.json 원문 정밀도를 유지했다. SupCon 비교용 기준선과 과거 v1은 같은 이름의 B1·256이라도 학습 경로가 다르다. 최종 후보 판단에는 해당 비교 설계와 비용도 함께 사용했다.

27 최종 선정 이력과 고정 Test

선정 이력	Test Accuracy	Test Macro F1
Skin v1	0.9871428571428571	0.9871314132317626
Web Skin v1	0.8825	0.8813822927981046
Web Skin v2	0.915	0.9141049081029712
Hair5 v1	0.7883386581469649	0.7885764577048346
Hair5 v2	0.8003194888178914	0.8002222282821301
Hair6 성능 우선	0.7915708812260537	0.8091500773460419
Hair6 경량	0.7869731800766283	0.8021613486071133

선정 이유를 읽는 순서

Skin은 정제 후 증강 B0·224, Web Skin은 일반 B0·256 v1에서 PMG B0·256 v2, Hair5는 B1·256 v1에서 B1·384 Adam v2로 이어졌다. Hair6는 별도 데이터 과제에서 B1·384 원본을 성능 우선, B0·256 증강을 경량 후보로 고정했다.

큰 PMG B1·384는 더 높은 Validation을 보였지만 입력·모델 크기와 실행 비용을 함께 고려해 B0를 남겼다. Hair6에서도 단일 절대 승자 대신 성능과 실행 비용의 두 역할을 보존했다. 이는 Test를 보고 계속 설정을 바꾸는 절차와 구분된다.

학습 조건 36개와 측정 조건 34개

학습·추가학습을 항목으로 세면 Skin 4, Web Skin 13, Hair5 15, Hair6 4의 총 36개다. Hair와 Web Skin의 추가 5회 이력을 별도 실행시간 조건으로 세지 않아 저장 모델 벤치마크는 34조건이다. 앙상블 확률 평균은 기존 예측을 결합한 분석이며 새 모델 학습을 한 번 더 센 항목이 아니다.

28 데이터·증강·평가 기록은 코드의 일부

학습용 데이터	원본 Train	증강 포함 Train	원본 Val / Test
Skin 초기	7,300	14,600	1,000 / 700
Skin Clean	7,249	14,498	1,000 / 700
Web Skin	3,600	7,200	500 / 400
Hair5 초기	11,600	17,400	1,450 / 1,450
Hair5 Clean	10,032	15,047	1,252 / 1,252
Hair6	10,460	17,615	1,305 / 1,305

초기 Skin 증강 포함 14,600장은 2026-10-27 당시 Colab 출력과 모델 해시로 확인한 최신 값이다. Skin/Hair의 정제 전후는 분할과 사진 구성이 달라 단순한 모델 향상률로 점수를 빼지 않는다. 모든 버전의 증강은 Train에 추가했고 Val/Test에는 생성 증강본을 더하지 않았다.

오프라인 증강과 학습 중 변환

초기 전처리는 회전·반전·밝기·대비·색·흐림·노이즈 등의 변형 사진을 파일로 저장하는 오프라인 증강이었다. SupCon은 여기에 두 view의 온라인 변환을, PMG는 jigsaw를, WS-DAN은 attention crop/drop을 학습 중 추가한다. MixStyle은 이미지 파일이 아니라 특징 통계를 바꾼다.

재현에 필요한 저장물

데이터 버전·SHA-256, 분할, 클래스 순서, 코드 commit과 실제 소스, 환경, 설정, 출발 가중치, best/last 체크포인트, 학습 이력, 선택된 모델의 평가·예측을 함께 남긴다. .keras는 모델, history.json은 학습 숫자, predictions.csv는 이미지별 결과다. 역할이 다르다.

29 실제 구현에서 확인한 네 가지 구분

1. SupCon 분류 단계에도 본체 일부가 학습된다

feature_encoder에 새 classifier를 붙인 뒤 trainable=False로 바꾸는 줄이 없다. 후반부 부분 학습 상태가 유지된다. DINOv2와 MedSigLIP처럼 고정 특징 위 Linear만 학습한 실험과 구분한다.

2. PMG Train Accuracy도 네 출력 합산이다

fusion_step의 반환값은 softmax(add_n(outputs))다. Train Accuracy와 Validation Accuracy는 합산 예측 기준이고, Loss는 각각 여러 항의 합과 fusion 단독 CE로 다르다. 일부 이전 문서의 Train Accuracy=fusion 단독 서술은 바로잡아 읽는다.

3. trainable=False와 training=False는 같은 말이 아니다

trainable은 optimizer가 바꿀 가중치 목록을 제어한다. training=False는 Dropout·BatchNormalization 등의 호출 동작을 제어한다. training=False로 호출한 본체라도 trainable 가중치가 있으면 gradient를 통해 학습될 수 있다. 반대로 BN 통계를 고정한 채 합성곱 일부를 학습하는 것이 가능하다.

4. 논문 이름을 import한 것과 아이디어 구현은 다르다

CE·LS·Focal은 Keras 제공 손실, B0/B1/V2-S는 Keras Applications, DINOv2는 KerasHub preset, MedSigLIP은 Hugging Face Transformers 모델을 사용했다. WS-DAN·PMG·MixStyle·SAM과 SupCon 학습 loop의 핵심은 프로젝트 코드다.

현재 코드는 실행 호환성 조치나 모델 선택 확장으로 초기 보존 소스와 다를 수 있다. 다음 페이지에 그 차이를 적었다. 코드에 어떤 옵션이 있다는 사실만으로 그 옵션이 과거 모든 실행에 적용됐다고 소급하지 않는다.

30 코드의 출처와 당시 소스 비교

현재 파일	주요 함수·역할	당시 저장 코드와 관계
common_engine.py	loss_function, build_model, configure_partial, fit_stage, extend_b1	공통 학습 동작을 설명하는 기준
paper_suite.py	_run_supcon, _build_dinov2_classifier	paper_screen 보존 파일과 구조 동일. SupCon 보존본의 핵심 3함수도 AST 동일
hair_sam.py	_fit_sam	sam_screen 보존 파일과 AST 동일
web_skin_wsdan.py	build_model, _fit_stage, AttentionEnsemblePredictor	두 Web Skin 보존 소스와 AST 동일
web_skin_paper_suite.py	PMG, MixStyle	초기 PMG는 B0 고정. 현재는 B0/B1 선택. _fit_pmg_stage와 MixStyle Layer는 초기 핵심 동작 동일
web_skin_medsiglip_linear.py / hair_medsiglip_linear.py	_load_encoder, _extract_split, _linear_probe	각 도메인의 당시 보존 파일과 AST 동일
hair_six_class_suite.py	dataset_factory, run_four	6클래스의 가중치·설정·선정 규칙 확인

AST 비교는 Python의 코드 구조를 비교하는 정적 점검이다. 모델을 다시 실행해 같은 수치가 나왔다는 뜻은 아니다. 현재 MixStyle 생성자에는 초기 보존 소스에 없던 jit_compile=False가 있어 구분했다. 이 문서의 발췌는 현재 src 경로와 줄 번호를 명시했다.

로컬 경로 기준은 프로젝트 루트이며 본체는 src/mediflow_datasets/, 당시 스냅샷은 results/<domain>/experiments/<run>/에 있다. 예: Hair paper_screen_20260917_141641_874f03f1, Web Skin web_skin_paper_suite_20260922_124758_717b465e.

31 논문과 공식 문서

원리·방법의 근거 링크다. 논문의 성능 수치를 MediFlow 결과로 가져오지 않았다. 코드 적용 범위와 결과는 앞의 실제 구현·저장 JSON을 기준으로 읽는다.

R01 · Label Smoothing: Rethinking the Inception Architecture for Computer Vision
https://arxiv.org/abs/1512.00567

R02 · Keras 교차 엔트로피·LS·Focal API
https://keras.io/api/losses/probabilistic_losses/

R03 · Focal Loss for Dense Object Detection
https://arxiv.org/abs/1708.02002

R04 · EfficientNet: Rethinking Model Scaling
https://proceedings.mlr.press/v97/tan19a.html

R05 · EfficientNetV2: Smaller Models and Faster Training
https://proceedings.mlr.press/v139/tan21a.html

R06 · 피부 병변의 Multi-Resolution EfficientNets 연구
https://arxiv.org/abs/1910.03910

R07 · Supervised Contrastive Learning
https://arxiv.org/abs/2004.11362

R08 · DINOv2: Learning Robust Visual Features without Supervision
https://arxiv.org/abs/2304.07193

R09 · Sharpness-Aware Minimization
https://arxiv.org/abs/2010.01412

R10 · WS-DAN: See Better Before Looking Closer
https://arxiv.org/abs/1901.09891

R11 · Progressive Multi-Granularity Training of Jigsaw Patches
https://arxiv.org/abs/2003.03836

R12 · MixStyle Neural Networks
https://arxiv.org/abs/2107.02053

R13 · Google MedSigLIP-448 모델 카드
https://huggingface.co/google/medsiglip-448

R14 · Keras 전이학습·미세조정 가이드
https://keras.io/guides/transfer_learning/

32 근거 파일과 재검토 방법

발췌한 현재 코드 파일의 SHA-256이다. 해시는 파일 내용이 같은지 확인하는 값이며, 아래 기록은 이 PDF를 작성할 때 읽은 소스 상태를 가리킨다.

src/mediflow_datasets/ 파일	SHA-256
common_engine.py	fc007c0ee29cd61f014c2c56c9623ed870c177480698de759c9548c211d174c7
common_workflow.py	68dd24cd34e810379002e190a17bfdbe9fc9ee9954061ab077549fe77dbe45b3
paper_suite.py	dcec51a0a7f099db930a4a1ce978caa124bbeb2798ec16c21358e1012fc5302b
hair_sam.py	f5192284b8109c140c5bdd0b0419d1f3c1527cf56cecb28552d8c202117eb844
web_skin_wsdan.py	4ade72232103b81d3ef2cd8b7827e26fa3eabc18e69a0b1dffccf1823a16fb84
web_skin_paper_suite.py	da0155994262ea24a97f834104f435ee400dd4a595be2e53b07e9ad0c06c574f
web_skin_medsiglip_linear.py	e8a27f9b123c2bd737bb512d08af391c97e4478d42992461b4ab057d465f141f
hair_six_class_suite.py	1ba2978cb82ab91ab8170b6675048e651d67683f8046faf0db8f8dfc7edb7b28

결과 수치 원본

Hair SupCon·paper_screen·SAM·MedSigLIP, Web Skin suite·paper_suite·PMG B1·MedSigLIP의 validation_metrics.json을 직접 읽었다. 최종 Test 표는 results/<domain>/selected_models/<version>/selection.json의 값을 그대로 사용했다. 이력의 최고 Accuracy, 마지막 Accuracy, 선정 체크포인트 재평가를 혼합하지 않는다.

연구 설명 원본

docs/research/ALL_EXPERIMENTS_RATIONALE_AND_RESULTS_20260924.md, HAIR_5CLASS_6CLASS_EXPERIMENT_ANALYSIS_20260928.md와 각 방법 결과 문서를 대조했다. 실행 단계는 docs/ROADMAP.md, 초기 연구 배경은 docs/research/PROJECT_BACKGROUND.md를 따른다.

본 PDF는 기존 모델을 바꾸지 않은 코드 해설이다. 다음 작업은 이 해설을 발표의 방법·구현 설명에 사용하고, 구현 변경이 필요할 때 해당 함수·저장 설정에서 출발하는 것이다.